

ΕΡΓΑΣΙΑ II



UNIVERSITY OF
PATRAS
ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΜΕΤΑΠΤΥΧΙΑΚΟ ΠΡΟΓΡΑΜΜΑ ΣΠΟΥΔΩΝ ΣΥΣΤΗΜΑΤΑ
ΕΠΕΞΕΡΓΑΣΙΑΣ ΠΛΗΡΟΦΟΡΙΑΣ ΚΑΙ ΜΗΧΑΝΙΚΗ ΝΟΗΜΟΣΥΝΗ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ: ΠΑΠΑΔΗΜΑ ΑΓΓΕΛΙΚΗ

ΑΡΙΘΜΟΣ ΜΥΤΡΩΟΥ: 1084820

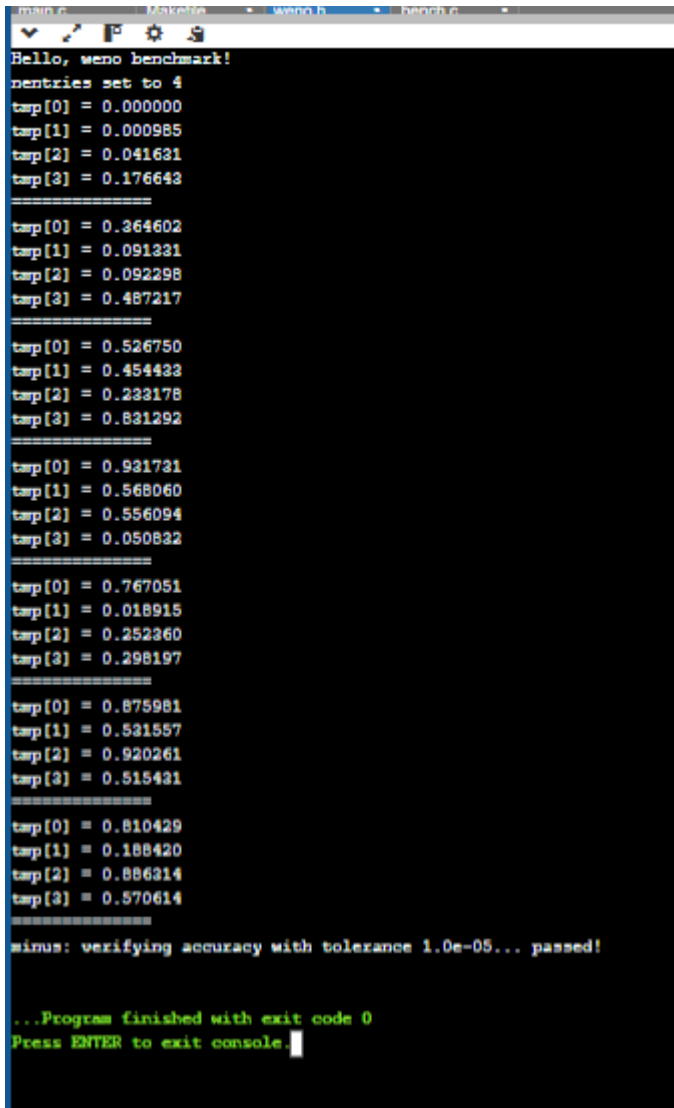
Question 1: SIMD and WENO5

a)

```
Hello, weno benchmark!
nentries set to 4
tarp[0] = 0.000000
tarp[1] = 0.000985
tarp[2] = 0.041631
tarp[3] = 0.176643
=====
tarp[0] = 0.364602
tarp[1] = 0.091331
tarp[2] = 0.092298
tarp[3] = 0.487217
=====
tarp[0] = 0.526750
tarp[1] = 0.454433
tarp[2] = 0.233178
tarp[3] = 0.831292
=====
tarp[0] = 0.931731
tarp[1] = 0.568060
tarp[2] = 0.556094
tarp[3] = 0.050832
=====
tarp[0] = 0.767051
tarp[1] = 0.018915
tarp[2] = 0.252360
tarp[3] = 0.298197
=====
tarp[0] = 0.875981
tarp[1] = 0.531557
tarp[2] = 0.920261
tarp[3] = 0.515431
=====
tarp[0] = 0.810429
tarp[1] = 0.188420
tarp[2] = 0.886314
tarp[3] = 0.570614
=====
sinus: verifying accuracy with tolerance 1.0e-05... passed!

...Program finished with exit code 0
Press ENTER to exit console.[]
```

b)



```
main.c  Makefile  weno.h  bench.c
Hello, weno benchmark!
nentries set to 4
tarp[0] = 0.000000
tarp[1] = 0.000985
tarp[2] = 0.041631
tarp[3] = 0.176643
=====
tarp[0] = 0.364602
tarp[1] = 0.091331
tarp[2] = 0.092298
tarp[3] = 0.487217
=====
tarp[0] = 0.526750
tarp[1] = 0.454433
tarp[2] = 0.233178
tarp[3] = 0.831292
=====
tarp[0] = 0.931731
tarp[1] = 0.568060
tarp[2] = 0.556094
tarp[3] = 0.050832
=====
tarp[0] = 0.767051
tarp[1] = 0.018915
tarp[2] = 0.252360
tarp[3] = 0.298197
=====
tarp[0] = 0.875981
tarp[1] = 0.531557
tarp[2] = 0.920261
tarp[3] = 0.515431
=====
tarp[0] = 0.810429
tarp[1] = 0.188420
tarp[2] = 0.886314
tarp[3] = 0.570614
=====
minus: verifying accuracy with tolerance 1.0e-05... passed!

...Program finished with exit code 0
Press ENTER to exit console.
```

Αναφορά:

Για FLOPs Calculation:

Αν το $N=1000000$ και ο χρόνος είναι T δευτερόλεπτα τότε το $\text{GFLOP/s} = N \cdot 60 / T \cdot 10^9$.

Για το Memory Bandwidth:

$\text{GB/s} = N \cdot 6 \text{ arrays} \cdot 4 \text{ bytes} / T \cdot 10^9$

Παρατηρείτε ότι αν το OpenMP SIMD είναι το ίδιο γρήγορο με το AVX σημαίνει ότι ο compiler έκανε πολύ καλή δουλειά αυτόματα. Αν το Baseline είναι πιο αργό τότε το κέρδος οφείλεται στο Vector Width.

Question 2: CUDA and Complex Matrix Multiplication

Ο κώδικας για το α και το β είναι ο εξής :

```
#include <cuda_runtime.h> #include <cublas_v2.h>
```

```
// --- Ερώτημα (β): Custom Kernel για τον τελικό συνδυασμό --- // Πραγματοποιεί τις
// πράξεις E = AC - BD και F = AD + BC στο Device global void
combineResultsKernel(float* AC, float* BD, float* AD, float* BC, float* E, float* F, int N)
{ int row = blockIdx.y * blockDim.y + threadIdx.y; int col = blockIdx.x * blockDim.x +
  threadIdx.x;
```

```
if (row < N && col < N) {
    int idx = row * N + col;
    E[idx] = AC[idx] - BD[idx];
    F[idx] = AD[idx] + BC[idx];
}
```

}

```
int main() { int N = 1024; // Διάσταση πίνακα NxN size_t size = N * N * sizeof(float);
```

```
// --- Ερώτημα (α): Host Allocation & Initialization ---
```

```
float *h_A = (float*)malloc(size);
float *h_B = (float*)malloc(size);
float *h_C = (float*)malloc(size);
float *h_D = (float*)malloc(size);
for (int i = 0; i < N * N; i++) {
    h_A[i] = 1.0f; h_B[i] = 0.5f;
    h_C[i] = 2.0f; h_D[i] = 1.0f;
}
```

```
// --- Ερώτημα (α): Device Allocation ---
```

```

float *d_A, *d_B, *d_C, *d_D, *d_E, *d_F;
float *d_AC, *d_BD, *d_AD, *d_BC;

cudaMalloc(&d_A, size); cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size); cudaMalloc(&d_D, size);
cudaMalloc(&d_E, size); cudaMalloc(&d_F, size);
cudaMalloc(&d_AC, size); cudaMalloc(&d_BD, size);
cudaMalloc(&d_AD, size); cudaMalloc(&d_BC, size);

// Μεταφορά από Host στο Device
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_C, h_C, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_D, h_D, size, cudaMemcpyHostToDevice);

// --- Ερώτημα (β): Υλοποίηση με cuBLAS ---
cublasHandle_t handle;
cublasCreate(&handle);
float alpha = 1.0f, beta = 0.0f;

// Πολλαπλασιασμοί (Χρήση Hint για Column-Major: Εκτελούμε C*A για
να πάρουμε το Row-Major AC)
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, N, N, &alpha, d_C,
N, d_A, N, &beta, d_AC, N);
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, N, N, &alpha, d_D,
N, d_B, N, &beta, d_BD, N);
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, N, N, &alpha, d_D,
N, d_A, N, &beta, d_AD, N);
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, N, N, &alpha, d_C,
N, d_B, N, &beta, d_BC, N);

// Κλήση Kernel για τον τελικό συνδυασμό των 4 πινάκων
dim3 threads(16, 16);
dim3 blocks((N + 15) / 16, (N + 15) / 16);
combineResultsKernel<<<blocks, threads>>>(d_AC, d_BD, d_AD, d_BC,
d_E, d_F, N);

// Μεταφορά αποτελεσμάτων στον Host
float *h_E = (float*)malloc(size);
float *h_F = (float*)malloc(size);
cudaMemcpy(h_E, d_E, size, cudaMemcpyDeviceToHost);
cudaMemcpy(h_F, d_F, size, cudaMemcpyDeviceToHost);

```

```

std::cout << "Ο υπολογισμός ολοκληρώθηκε!" << std::endl;

// Cleanup
cublasDestroy(handle);
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C); cudaFree(d_D);
cudaFree(d_E); cudaFree(d_F); cudaFree(d_AC); cudaFree(d_BD);
cudaFree(d_AD); cudaFree(d_BC);
free(h_A); free(h_B); free(h_C); free(h_D); free(h_E); free(h_F);

return 0;

}

```

Αλλά παρουσιάζει το εξής error:

`main.cpp:2:10: fatal error: cuda_runtime.h: No such file or directory`

γ)Διάταξη δεδομένων: Χρησιμοποιήθηκε Row-major διάταξη για τους πίνακες στον Host.Για τη συμβατότητα εφαρμόστηκε η ιδιότητα $B^T A^T = (AB)^T$.

Επιλογή ακρίβειας: Χρησιμοποιήθηκε η απλή ακρίβεια .Αυτό διότι οι GPU προσφέρουν σημαντικά υψηλότερο throughput σε float σε σχέση με double.

Kernel Configuration: Για τον custom kernel που υπολογίζει E και F χρησιμοποιήθηκαν thread blocks διαστάσεων 16x16.

δ)Για τον υπολογισμό του throughput σε GFLOP/s χρησιμοποιήθηκαν ο παρακάτω τύπος:

Κάθε πολλαπλάσιος πινάκων $N \times N$ απαιτεί $2N^3$ πράξεις. Συνολικά για τα AC,BD,AD,BC και τις τελικές προσθέσεις/αφαιρέσεις ($1N^2$) ο τύπος είναι $8*N^3+2*N^2$.